

5) MISSIONARIES CANNAIBAL PROBLEM

Aim

To develop a Python program to solve the Missionaries and Cannibals Problem using the Breadth-First Search (BFS) algorithm.

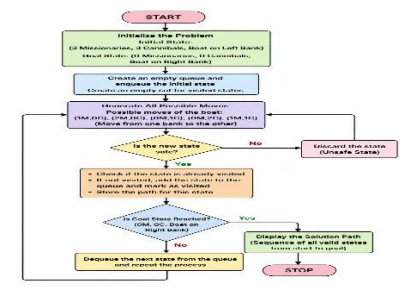
Objectives

- To understand the Missionaries and Cannibals problem in Artificial Intelligence.
- To apply the Breadth-First Search (BFS) algorithm to find a valid solution.
- To transport all missionaries and cannibals safely across the river.
- To ensure that missionaries are never outnumbered by cannibals on either riverbank.

Algorithm

1. Start the program.
2. Represent the initial state as (3 Missionaries, 3 Cannibals, Boat on Left).
3. Generate all possible valid boat movements.
4. Check whether each generated state is safe.
5. Ignore invalid or previously visited states.
6. Continue searching until the goal state (0, 0, Boat on Right) is reached.
7. Display the sequence of valid states.
8. Stop the program.

Flow Chart



Python Program

```
from collections import deque
```

```
def is_safe(m, c):
```

```
    return (m == 0 or m >= c) and (3 - m == 0 or 3 - m >= 3 - c)
```

```
def bfs():
```

```
    start = (3, 3, 1)
```

```
    goal = (0, 0, 0)
```

```
    queue = deque([(start, [start])])
```

```
    visited = set()
```

```
    while queue:
```

```
        (m, c, b), path = queue.popleft()
```

```
        if (m, c, b) == goal:
```

```
            return path
```

```
        if (m, c, b) in visited:
```

```
            continue
```

```
        visited.add((m, c, b))
```

```
        moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]
```

```
        for dm, dc in moves:
```

```
            if b == 1:
```

```
                nm, nc, nb = m-dm, c-dc, 0
```

```
            else:
```

```
                nm, nc, nb = m+dm, c+dc, 1
```

```
            if 0 <= nm <= 3 and 0 <= nc <= 3 and is_safe(nm, nc):
```

```
                queue.append(((nm, nc, nb), path+[(nm, nc, nb)]))
```

```
    solution = bfs()
```

```
    print("Solution Path:")
```

```
    for state in solution:
```

```
        print(state)
```

Sample Output

```
Output
Solution Path:
(0, 1, 1)
(1, 1, 0)
(1, 2, 1)
(1, 0, 0)
(0, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 2, 1)
(0, 1, 0)
(1, 1, 1)
(0, 0, 0)
... Code Execution Successful ...
```

Result

Thus, the Python program to solve the Missionaries and Cannibals Problem using the Breadth-First Search (BFS) algorithm was implemented successfully, and all missionaries and cannibals were transported safely across the river.

Conclusion

The Missionaries and Cannibals Problem demonstrates the application of the Breadth-First Search (BFS) algorithm in Artificial Intelligence. The algorithm systematically explores all valid states while satisfying the problem constraints and successfully finds a safe sequence of moves to reach the goal state.

6) VACCUM CLEANER PROBLEM

Aim

To develop a Python program to solve the Vacuum Cleaner Problem using a simple Artificial Intelligence agent.

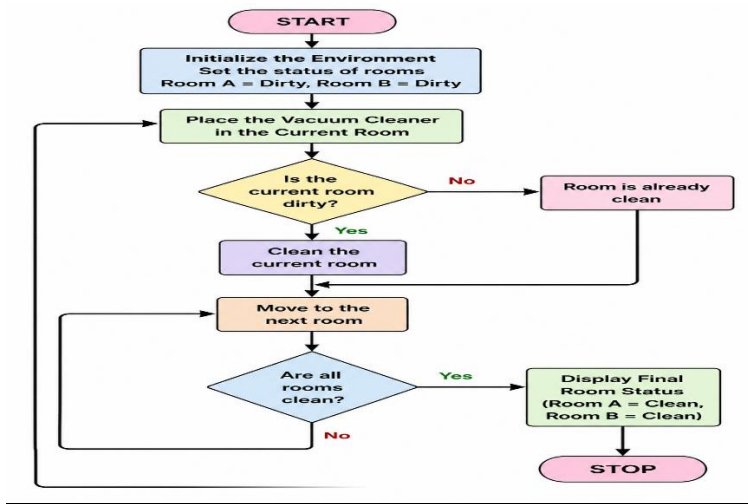
Objectives

- To understand the working of a simple reflex agent.
- To simulate the behavior of a vacuum cleaner in a two-room environment.
- To clean all dirty rooms efficiently.
- To demonstrate the concept of intelligent agents in Artificial Intelligence.

Algorithm

1. Start the program.
2. Read the status of Room A and Room B.
3. Place the vacuum cleaner in the initial room.
4. Check whether the current room is dirty.
5. If dirty, clean the room.
6. Move to the next room.
7. Repeat the cleaning process.
8. Stop when both rooms are clean.

Flow Chart



Python Program

Vacuum Cleaner Problem

```
rooms = {'A': 'Dirty', 'B': 'Dirty'}
```

```
for room in rooms:
```

```
    print("Vacuum is in Room", room)
```

```
    if rooms[room] == "Dirty":
```

```
        print("Cleaning Room", room)
```

```
rooms[room] = "Clean"
```

```
else:
```

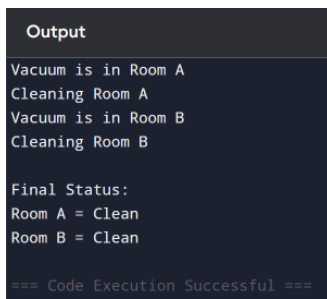
```
    print("Room is already Clean")
```

```
print("\nFinal Status:")
```

```
for room in rooms:
```

```
    print("Room", room, "=", rooms[room])
```

Sample Output



```
Output
Vacuum is in Room A
Cleaning Room A
Vacuum is in Room B
Cleaning Room B

Final Status:
Room A = Clean
Room B = Clean

=== Code Execution Successful ===
```

Result

Thus, the Python program to solve the Vacuum Cleaner Problem was implemented successfully, and both rooms were cleaned.

Conclusion

The Vacuum Cleaner Problem demonstrates the behavior of a simple reflex agent in Artificial Intelligence. The agent senses the environment, performs the appropriate action, and successfully cleans all dirty rooms.